Лабораторная работа №3. Транспонирование матрицы
Вариант A: double, $N_v = 3000$

Характеристики системы

Apple M2: 8 физических / 8 логических CPU, кэш L1 128 КБ (I) / 64 КБ (D), L2 4 МБ, L3/SLC — унифицированный, не отображается. Компилятор gcc-16 (Homebrew GCC 16.2.0), ключи `-O2 -fopenmp`. Время — `omp_get_wtime()`, медиана 5 запусков после прогрева; BW = $2N^2 \cdot s$ / t, s = 8 байт; все версии сверены с `transpose_rows` (`check=OK`).

Эксперимент 1. Влияние порядка обхода

Один поток, `transpose_rows` vs `transpose_cols`.

N	Время rows, с	Время cols, с	BW rows, ГБ/с	BW cols, ГБ/с
256	0.000177	0.000044	5.92	23.59
512	0.000852	0.000231	4.92	18.12
1024	0.004532	0.000987	3.70	17.00
2048	0.024328	0.013871	2.76	4.84
3000	0.009086	0.014473	15.85	9.95

Эксперимент 2. Влияние размера блока

Один поток, N = 3000, `transpose_blocked`.

b	4	8	16	32	64	128
Время, с	0.008454	0.009797	0.007145	0.008567	0.009688	0.009234
BW, ГБ/с	17.03	14.70	20.15	16.81	14.86	15.59

Минимум — **b* = 16** (0.007145 с, 20.15 ГБ/с).

Эксперимент 3. Влияние количества потоков

N = 3000, b = b* = 16. Ускорение $S = T_1 / T_p$ относительно однопоточной версии той же функции.

Потоки	Время rows_omp, с	S rows_omp	Время blocked_omp, с	S blocked_omp
1	0.009127	1.00	0.007171	1.00
2	0.004886	1.87	0.009153	0.78
4	0.003199	2.85	0.005636	1.27
8	0.003154	2.89	0.004494	1.60

Эксперимент 4. False sharing

8 потоков, N = 3000, `transpose_rows_omp` с разными стратегиями.

Стратегия	schedule(static)	schedule(static, 1)	schedule(static, 8)
Время, с	0.003133	0.008172	0.004177
BW, ГБ/с	45.97	17.62	34.47

Вывод

Rows vs cols. В `transpose_rows` A читается последовательно (локальность и префетчинг), B пишется с шагом N; в `transpose_cols` — наоборот. При N = 256–2048 выигрывает последовательная запись (cols быстрее в 2–4 раза). При N = 3000 (≈ 72 МБ, в 18 раз больше L2) решает последовательное чтение: rows обгоняет cols (15.85 против 9.95 ГБ/с), то есть разница заметна уже при $N \geq 512$, а смена лидера — около $N \approx 3000$.

Связь b* с кэшами. b* = 16 даёт подматрицу 16×16 = 2048 байт, которая с большим запасом помещается в 64 КБ L1d и 4 МБ L2: чтение и запись остаются в кэше на всём протяжении блока. Блоки 32–128 теряют локальность, 4–8 — не дают преимуществ блочного метода.

Ограничение ускорения. Транспонирование — memory-bound: на элемент одно чтение и одна запись, ноль арифметики, поэтому упирается в общую пропускную способность памяти плюс когерентность кэшей. В ЛР2 (compute-bound) ядра считают независимо, поэтому S ≈ линейно; здесь S rows_omp(8) = 2.89. У blocked_omp при 2 потоках — даже замедление (S = 0.78): при b = 16 блоки мелкие, и накладные расходы OpenMP превышают выигрыш; масштабирование начинается с 4 потоков.

False sharing. При `schedule(static, 1)` потоки пишут в B[jN+i] и B[jN+i+1] соседних строк — обе ячейки в одной 64-байтной кэш-линии (8 double), линия «перебрасывается» между ядрами: время растёт с 0.003133 до 0.008172 с. При `schedule(static)` каждый поток получает непрерывный блок строк, области разнесены — false sharing нет; `schedule(static, 8)` — промежуточный случай (0.004177 с).